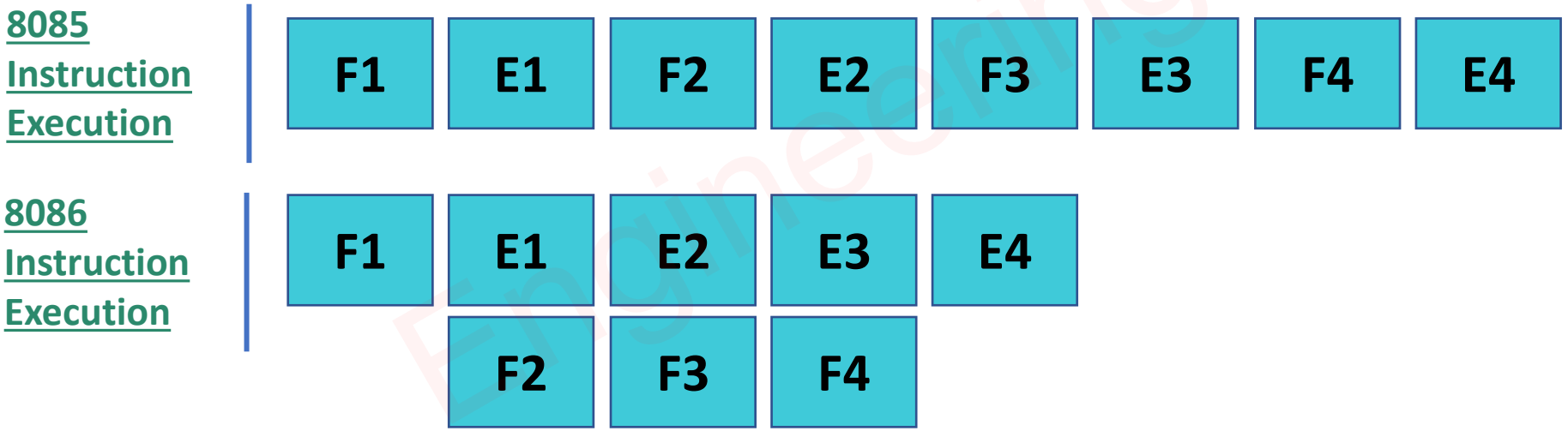
Pipelining in Computer Organization & Architecture

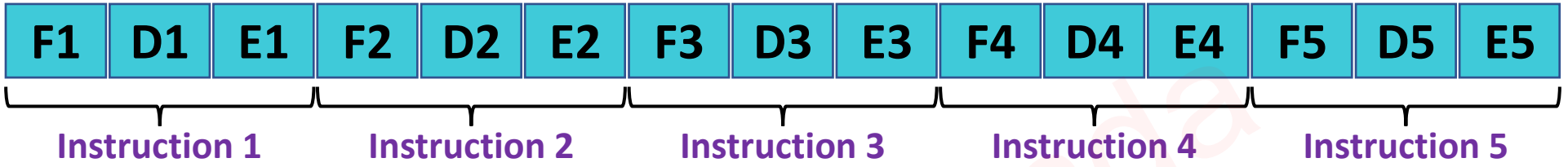
- Pipeline defines the overlapping of stages for instruction execution.
- Pipeline is done to improve execution speed of programs.

Example 1

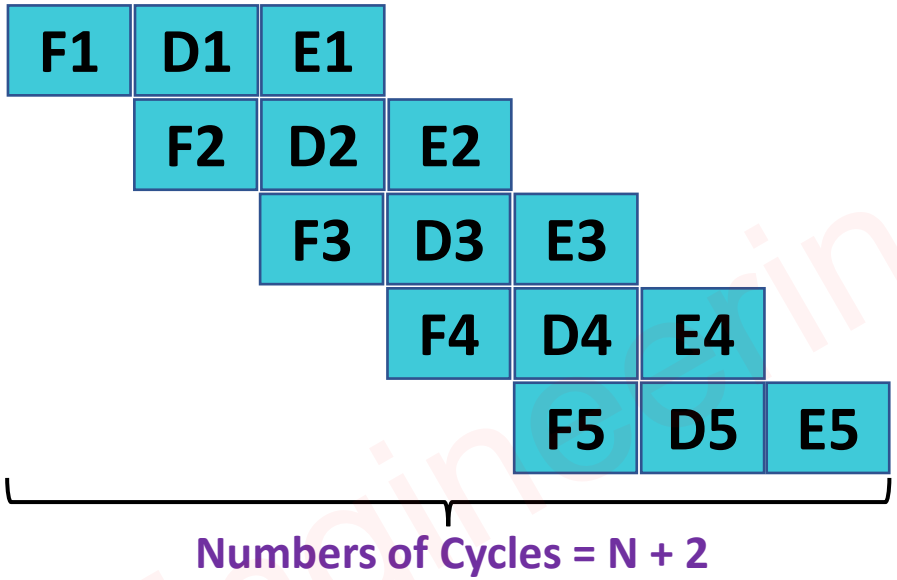


Example 2

Normal Execution without Pipelining



ARM7 with Pipelining

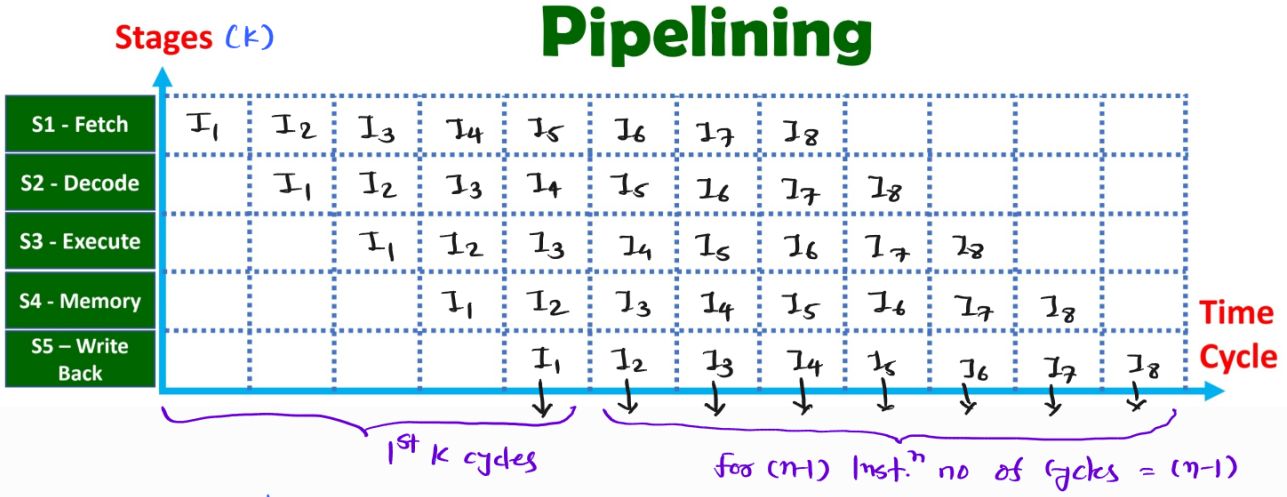


- **Advantage: It improves execution speed of program.**

Parameters of Pipelining in Computer Organization & Architecture

- Pipeline is used to improve speed of execution of program.
- Lets assume ARM9 architecture for 5 stage pipeline.
  - Fetch {Instruction Fetch}
  - Decode {Instruction Decode}
  - Execute {Instruction Execution}
  - Memory {Memory Write}
  - Write Back {Register Write}
- Lets have 8 instructions execution from I1 to I8.

Instruction Space time Diagram with Pipelining



→ $T_{wp} = nk\,t_c$

$T_p = (n+k-1)t_c$

→ Speed up $S = \frac{T_{wp}}{T_p} = \frac{nk\,t_c}{(n+k-1)t_c} = \boxed{\frac{nk}{(n+k-1)}}$

→ Throughput $= \frac{n}{(n+k-1)t_c} = \left(\frac{1}{t_c}\right)$ ← ideal

→ Utilization or Efficiency $\eta = \frac{nk}{(n+k-1)k} = \frac{n}{n+k-1}$

→ $\boxed{S = k\eta}$

Pipelining Hazards

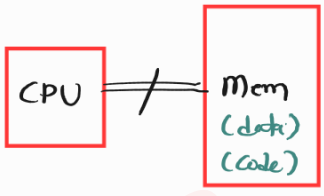
- During Pipelined execution of programs there can be three major categories of Hazards.
  1. Data Hazards
  2. Control Hazards
  3. Structural Hazards

Structural Hazards in Pipelining

→ Lets take four stage pipeline :

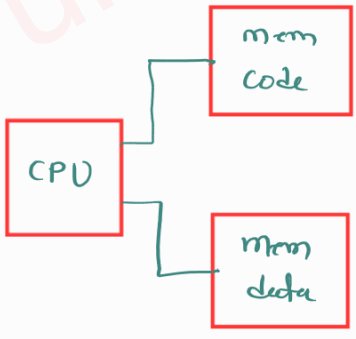
1. Instruction Fetch (IF)
2. Instruction decode + Operand decode (ID)
3. Execute Instruction (EX)
4. Writeback (WB)

	1	2	3	4	5	6	7
Mem. Inst.ⁿ → I_1	IF	ID	EX	WB ← Mem			
I_2		IF	ID	EX	WB		
I_3			IF	ID	EX	WB	
I_4				IF → Mem	ID	EX	WB



→ Sol.ⁿ - Havard Architecture.

	1	2	3	4	5	6	7
Mem. Inst.ⁿ → I_1	IF	ID	EX	WB ← Mem (data)			
I_2		IF	ID	EX	WB		
I_3			IF	ID	EX	WB	
I_4				IF → Mem (code)	ID	EX	WB



	1	2	3	4	5	6	7	8
(MUL) I_1	IF	ID	EX	EX	EX	WB		
(ADD) I_2		IF	ID	-	-	EX	WB	

↑ 2 clock cycles are stalled

Data Hazards in Pipelining

I_1 : ADD R_1, R_2, R_3 (R_2 + R_3 → R_1)

I_2 : ADD R_4, R_1, R_5 (R_1 + R_5 → R_4)

	1	2	3	4	5	6	7	8	9
I_1	IF	ID	EX	M	WB				
I_2		IF	ID	EX	M	WB			

Result of R_1

False result in R_4

→ Operand Forwarding is the soln.

	1	2	3	4	5	6	7	8	9
I_1	IF	ID	EX	M	WB				
I_2		IF	ID	EX	M	WB			

Result of R_1

Correct Result.

→ RAW - Read After write (Actual / True dependency)

- ADD R_1, R_2, R_3
- ADD R_4, R_1, R_5

$IO\{NI\} \cap OO\{1^{st}I\} \neq \phi$

R_1 : Input Operands

R_4 : Output Operands.

→ WAR - Write After Read (Anti dependency).

ADD R_1, R_2, R_3

ADD R_2, R_1, R_3

$IO\{1^{st}I\} \cap O.O.\{N.I\} \neq \phi$

R_1 : Input Operands

R_2 : Output Operands.

→ WAW - Write After write (Output dependency)

ADD R_1, R_2, R_3

ADD R_1, R_4, R_5

$OO\{1^{st}I\} \cap O.O\{N.I\} \neq \phi$

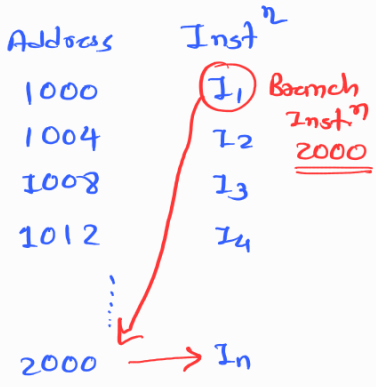
R_1 : Output Operands.

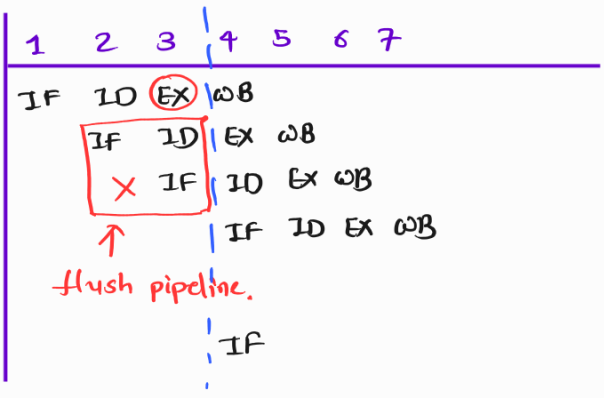
Control Hazards in Pipelining

→ Control Hazards in pipeline is happening due to branch Inst.ⁿ

→ Lets have four stage pipeline : IF , ID , EX , WB

IF → 1st stage, ID → 2nd stage, EX → 3rd stage, WB → 4th stage.





→ During branch no cycles stall = EX Stage − 1

→ Sol.ⁿ is branch prediction logic.

❖ Data Hazards

- ❑ It happens due to data dependency of instructions in pipeline structure.
- ❑ When output of 1st instruction is input of 2nd instruction at that time it happens.
- ❑ Due to such conditions, we need to flush the pipeline.

❖ Control Hazards

- ❑ It happens due to branch instructions in pipeline structure.
- ❑ During branch execution, all the instructions in pipeline should be removed as program control will be transferred to new location, where we need to fill the pipe again.
- ❑ To avoid such Hazards, we can implement branch prediction algorithm.

❖ Structural Hazards

- ❑ It happens due to resource dependency of instructions in pipeline structure.
- ❑ Like, during instruction execution we used same memory bus to store results.
- ❑ We avoid such Hazards, by implementation of Harvard Architecture or by using multiple resources.

Examples on Pipelining in COA

- **GATE CS – Consider a 5 stage pipeline with cycle time of 5ns. Calculate the execution time of 100 instructions and Speed up due to pipeline. Also find the utilization.**

→ $k = 5$

$t_c = 5ns$

$n = 100$

→ $T_p = (n+k-1)t_c = (100+5-1)5 = 520 nsec$

→ $S = \frac{nk}{n+k-1} = \frac{100 \times 5}{100+5-1} = 4.8077$

→ $\eta = \frac{n}{n+k-1} = \frac{100}{100+5-1} = 0.9615$

❑ **GATE CS – Consider a 5 stage pipeline. Delay of each stage is given as per 10ns, 16ns, 12ns, 11ns & 14ns, respectively. Calculate the execution time of 100 instructions and Speed up due to pipeline.**

→ $k = 5$

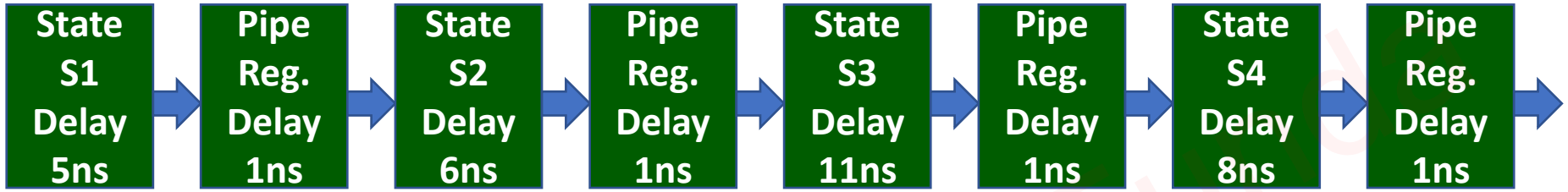
$t_c = 16\ nsec$

$n = 100$

→ $T_p = (n + k - 1)\, t_c = (100 + 5 - 1)\, 16 = 1664\ nsec = 1.664\ \mu sec$

→ $S = \dfrac{T_{wp}}{T_p} = \dfrac{100(10 + 16 + 12 + 11 + 14)}{1664} = 3.786$

- **GATE 2011 CS – Consider an instruction pipeline with four stages (S1, S2, S3 & S4) each with combinational circuit only. The pipeline registers are required between each stage and at the end of the last stage. Delay for the stages and for the pipeline registers are as given in the figure.**



- **What is the approximate speed up of the pipeline in steady state under ideal conditions when compared to the corresponding non pipeline implementation?**

$$S = \frac{T_{wp}}{T_p} = \frac{[5 + 6 + 11 + 8]}{[11 + 1]} = \underline{2.5}$$

Examples on Pipelining in COA

- **GATE CS – Consider a pipeline having 5 stages with duration 10ns, 30ns, 45ns, 80ns and 35ns. If the buffer delay is 20ns, calculate the speed up of pipelined processor.**

$$S = \frac{T_{wp}}{T_p} = \frac{[10 + 30 + 45 + 80 + 35]}{[80 + 20]} = 2$$

❑ **GATE CS – Assume we have two pipelines P1 and P2, respectively. P1 has 6 stages, having execution time of 12ns, 14ns, 19ns, 20ns, 22ns and 25ns. P2 has 4 stages each having execution time of 10ns. Calculate the time that can be saved while using P2 pipeline over P1 pipeline, If 2000 instructions are executed.**

→ P_1 , $K_1 = 6$, $t_{c_1} = 25$ nsec

→ P_2 , $K_2 = 4$, $t_{c_2} = 10$ nsec

→ $n = 2000$

→ $T_{P_1} = (n + K_1 - 1)\, t_{c_1} = (2000 + 6 - 1)\, 25 = 50125$ nsec

→ $T_{P_2} = (n + K_2 - 1)\, t_{c_2} = (2000 + 4 - 1)\, 10 = 20030$ nsec

→ $\Delta T = T_{P_1} - T_{P_2} = 50125 - 20030 = 30095$ nsec

❑ **GATE CS – Assume a pipeline P which operates at 3GHz clock rate. It has a speed up factor of 10 and efficiency of 40%. Calculate the number of stages in the above pipeline.**

→ $f_{clock} = 3\,GHz$

$S = 10$

$\eta = 40\% = 0.4$

→ $S = K\eta \Rightarrow K = \frac{10}{0.4} = 25$

Examples on Pipelining in COA

- **GATE 2015 CS – Consider a non pipelined processor with a clock rate of 2.5GHz and average cycles per instruction of four. The same processor is upgraded to a pipelined processor with five stages; but due to internal pipeline delay, the clock speed is reduced to 2GHz. Assume that there are no stalls in the pipeline. The speed up achieved in this pipelined processor is ……**

→ $f_{c_1} = 2.5\ GHz$, $CPI = 4$

→ $K = 5$, $f_{c_2} = 2\ GHz$

→ $S = \frac{T_{wp}}{T_p} = \frac{4/2.5 \times 10^9}{1/2 \times 10^9} = \frac{8}{2.5} = 3.2$

❑ **GATE CS – The stage delays in a 4 stage pipeline are 5ns, 6ns, 4ns and 5ns. The second stage is replaced with a functionally equivalent design involving two stages with respective delays 4ns and 5ns. The throughput increase of the pipeline is ……20…… %**

→ P-1 { 5ns, 6ns, 4ns, 5ns }

→ P-2 { 5ns, 4ns, 5ns, 4ns, 5ns }.

→ Throughput $= \frac{n}{(n+k-1)t_c} = \frac{1}{t_c}$ ← Ideally.

→ $t_{c_1} = 6ns$, $t_{c_2} = 5ns$

→ $TP_1 = \frac{1}{t_{c_1}} = \frac{1}{6ns}$

→ $TP_2 = \frac{1}{t_{c_2}} = \frac{1}{5ns}$.

→ $\%\uparrow_a = \frac{TP_2 - TP_1}{TP_1} \times 100$

$= \frac{1/5 - 1/6}{1/6} \times 100$

$= \frac{6}{5} - 1 = 0.2 \times 100 = 20\%$.

❑ **GATE 2004 CS – A 4 stage pipeline has the stage delays as 150, 120, 160 and 140 nanoseconds, respectively. Registers that are used between the stages have a delay of 5 ns each. Assuming constant clock rate, the total time taken to process 1000 data items on this pipeline will be ……. Microseconds**

$n = 1000$

$k = 4$

$t_c = 160 + 5 = 165$ nsec

$\rightarrow T_p = (n + k - 1) t_c$

$= (1000 + 4 - 1) 165$

$= 165495$ nsec

$= 165.495$ usec

- **GATE 2014 CS – Consider the following processors. Assume that the pipeline registers have zero latency.**
  - **P1: Four stage pipeline with stage latencies 1ns, 2ns, 2ns, 1ns.** $\Rightarrow t_{P_1} = 2\,nsec$
  - **P2: Four stage pipeline with stage latencies 1ns, 1.5ns, 1.5ns, 1.5ns.** $\Rightarrow t_{P_2} = 1.5\,nsec$
  - **P3: Five stage pipeline with stage latencies 0.5ns, 1ns, 1ns, 0.6ns, 1ns.** $\Rightarrow t_{P_3} = 1\,nsec$
  - **P4: Five stage pipeline with stage latencies 0.5ns, 0.5ns, 1ns, 1ns, 1.1ns.** $\Rightarrow t_{P_4} = 1.1\,nsec$
- **Which processor has the highest peak clock frequency?**

$$f_{P_3} = \frac{1}{t_{P_3}} = \frac{1}{1\,nsec} = 1\,GHz$$

$$f_{P_4} = \frac{1}{t_{P_4}} = \frac{1}{1.1\,nsec} = 0.909\,GHz$$

$$f_{P_2} = \frac{1}{t_{P_2}} = \frac{1}{1.5\,nsec} = 0.667\,GHz$$

$$f_{P_1} = \frac{1}{t_{P_1}} = \frac{1}{2\,nsec} = 0.5\,GHz.$$

Examples on Pipelining in COA

- **GATE 2009 CS** – Consider an instruction pipeline with five stages without any branch prediction: Fetch Instruction (FI), Decode Instruction (DI), Fetch Operand (FO), Execute Instruction (EI) and Write Operand (WO). The stage delay for FI, DI, FO, EI and WO are 5ns, 7ns, 10ns, 8ns and 6ns, respectively. There are intermediate storage buffer after each stage and the delay of each buffer is 1ns. A program consisting of 12 instructions I1, I2, I3, … , I12 is executed in this pipelined processor. Instruction I4 is only the branch instruction and its branch target is I9. If the branch is taken during the execution of this program, the time needed to complete the program is?

- $\frac{I_1, I_2, I_3, I_4, I_9, I_{10}, I_{11}, I_{12}}{n = 8}$

- FI, DI, FO, EI, WO

  ↑ 4th Stage.

- No of stalled Cycles = 4 − 1 = 3

- $T_p = t_p + t_s$

  $= (n + k - 1)\, t_c + x\, t_c$

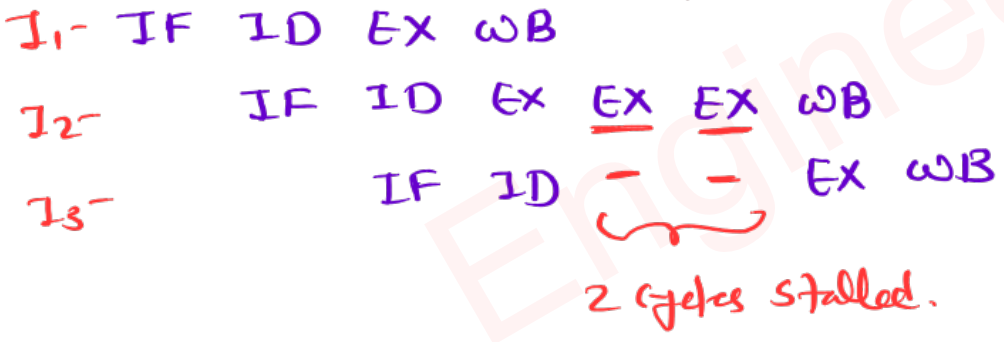
  $= (8 + 5 - 1)\, 11 + 3 \times 11$

  $= 15 \times 11$

  $= 165$ nsec

- **GATE CS – Consider the pipelined processor with the following four stages:**
  - **IF: Instruction Fetch**
  - **ID: Instruction Decode and Operand Fetch**
  - **EX: Execute**
  - **WB: Write Back**
- **The IF, ID and WB stages take one clock cycle each to complete the operation. The number of clock cycles for the EX stage depends on the instruction. The number ADD and SUB instructions need 1 clock cycle and MUL instruction needs 3 clock cycle in the EX stage. Operand forwarding is used in the pipelined processor. What is the number of clock cycles taken to complete the following sequence of instructions?** = 8

```
ADD R2, R1, R0      ;R2 ← R1 + R0
MUL R4, R3, R2      ;R4 ← R3 x R2
SUB R6, R5, R4      ;R6 ← R5 + R4
```



Cycles = Pipe Cycles + Hazards Cycles

= (n+k-1) + 1x2

= 3+4-1 +2 = 8

Examples on Pipelining in COA

GATE 2017 CS – Instruction execution in a processor is divided into 5 stages. Instruction Fetch (IF), Instruction Decode (ID), Operand Fetch (OF), Execute (EX), and Write Back (WB), These stages take 5, 4, 20, 10 and 3 nanoseconds (ns), respectively. A pipelined implementation of the processor requires buffering between each pair of consecutive stages with a delay of 2 ns. Two pipelined implementations of the processor are contemplated:
(i) a naïve pipeline implementation (NP) with 5 stages and
(ii) an efficient pipeline (EP) where the OF stage is divided into stages OF1 and OF2 with execution times of 12 ns and 8 ns respectively.
The speedup (correct to two decimals places) achieved by EP over NP in executing 20 independent instructions with no hazards is _____.

✓ (A) 1.50-1.51
(B) 1.51-1.52
(C) 1.52-1.53
(D) 1.53-1.54

NP: { 5, 4, 20, 10, 3 }

$k_1 = 5$

$t_{c_1} = 20 + 2 = 22$ nsec

EP: { 5, 4, 12, 8, 10, 3 }

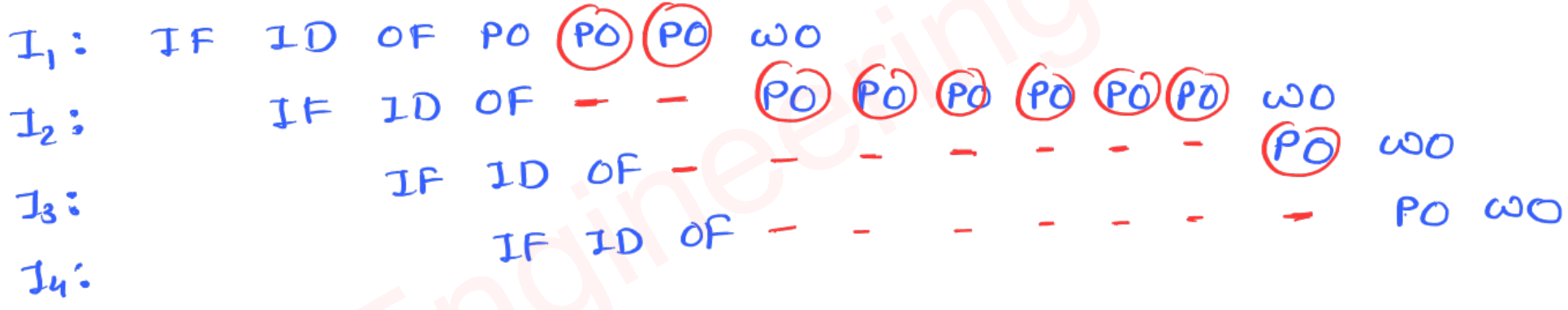
$k_2 = 6$

$t_{c_2} = 12 + 2 = 14$ nsec

$$\rightarrow S = \frac{T_{NP}}{T_{EP}} = \frac{(n + k_1 - 1)\, t_{c_1}}{(n + k_2 - 1)\, t_{c_2}} = \frac{(20 + 5 - 1)\, 22}{(20 + 6 - 1)\, 14} = 1.508$$

❑ **GATE 2010 CS – A 5 stage pipelined processor has instruction Fetch (IF), Instruction Decode (ID), Operand Fetch (OF), Perform Operation (PO) and write back (WO) stages. The IF, ID, OF and WO stages take 1 clock cycle each for any instruction. The PO stage take 1 cycle for ADD and SUB instructions, 3 clock cycles for MUL instruction and 6 clock cycles for DIV instruction respectively. Operand forwarding is used in the pipeline. What is the number of cycles needed to execute the following sequence of instruction?** (15)

I_1	(3)	MUL R2, R0, R1	;R2 ← R0 x R1
I_2	(6)	DIV R5, R3, R4	;R5 ← R3 / R4
I_3	(1)	ADD R2, R5, R2	;R2 ← R5 + R2
I_4	(1)	SUB R5, R2, R6	;R5 ← R2 – R6



→ Cycles = Normal Cycles + Hazards Cycles.

= (n+k-1) + (5+2)

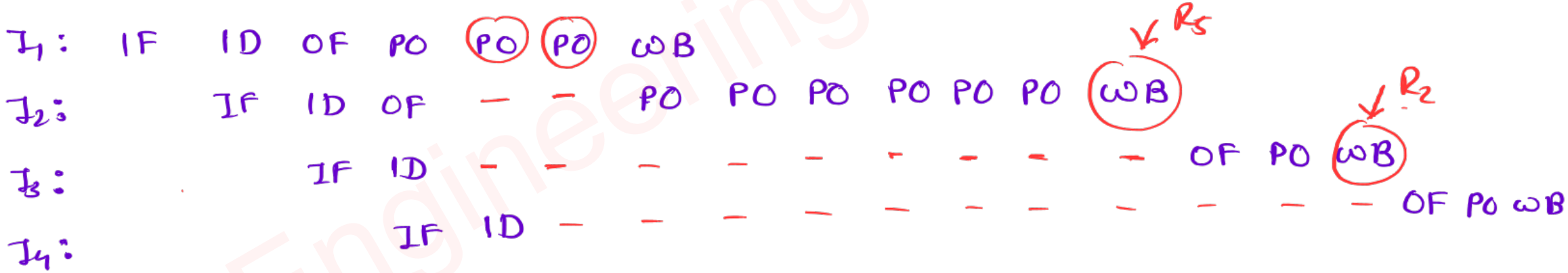
= (4+5-1) + 5+2

= (15)

- What is the number of cycles needed to execute the following sequence of instruction without data forwarding in above question? (19)

I_1	MUL R2, R0, R1 (3)	;R2 ← R0 x R1
I_2	DIV R5, R3, R4 (6)	;R5 ← R3 / R4
I_3	ADD R2, R5, R2 (1)	;R2 ← R5 + R2
I_4	SUB R5, R2, R6 (1)	;R5 ← R2 – R6

Output Operands. Input Operands



Examples on Pipelining Hazards in COA

GATE 2018 CS – The Instruction pipeline of a RISC processor has the following stages: Instruction (IF), Instruction Decode (ID), Operand Fetch (OF), Perform Operation (PO) and Writeback (WB). The IF, ID, OF and WB stages take 1 clock cycle each for every instruction. Consider a sequence of 100 instructions. In the PO stage, 40 instructions take 3 clock cycles each, 35 instructions take 2 clock cycles each and the remaining 25 instructions take 1 clock cycle each. Assume that there are no data hazards and no control hazards. The number of clock cycles required for completion of execution of the sequence of instruction is ……219……

→ No of Cycles = Normal pipeline Cycles + Cycles due to Structural Hazard

= (n + k − 1) + 35 × 1 + 40 × 2

= (100 + 5 − 1) + 35 + 80

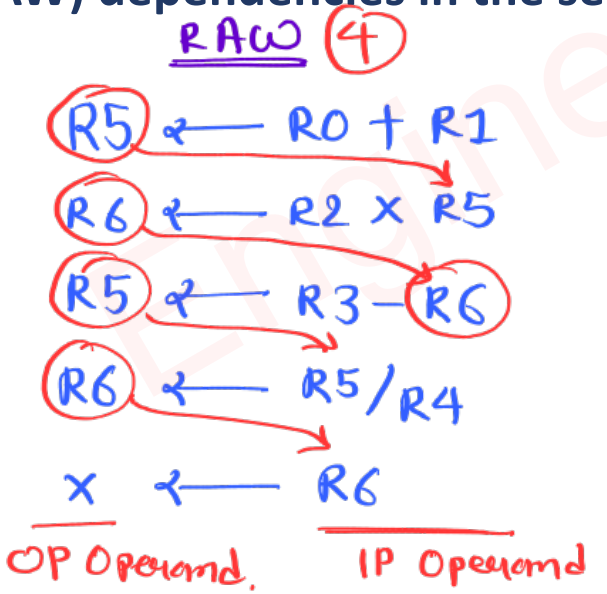
= 219

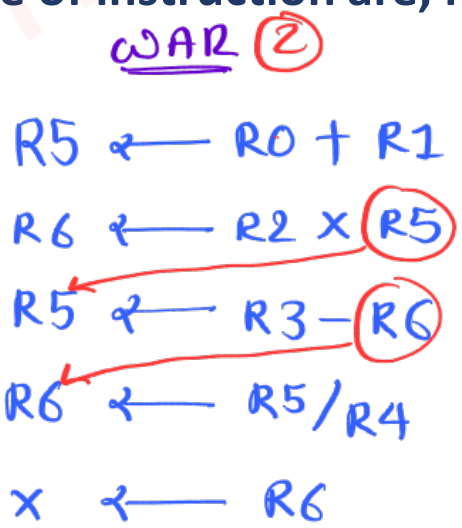
- **GATE 2006 CS** – A pipelined processor uses 4 stage instruction pipeline with the following stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX) and Writeback (WB). The arithmetic operations as well as the load and store operations are carried out in the EX stage. The sequence of instruction corresponding to the statement X = (S – R x (P + Q))/ T is given below. The values of variables P, Q, R, S and T are available in the registers R0, R1, R2, R3 and R4, respectively, before the execution of the instructions.

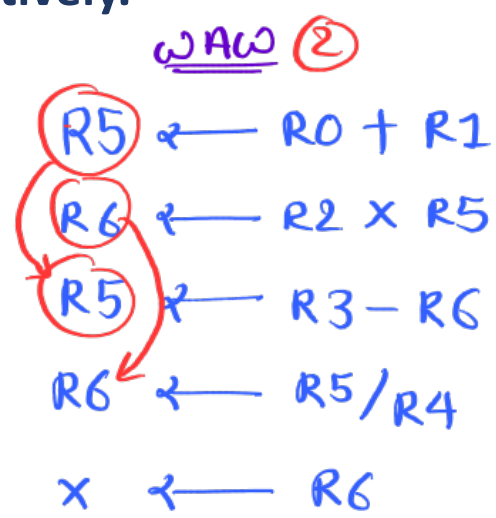
```
ADD R5, R0, R1    ;R5 ← R0 + R1
MUL R6, R2, R5    ;R6 ← R2 x R5
SUB R5, R3, R6    ;R5 ← R3 – R6
DIV R6, R5, R4    ;R6 ← R5 / R4
STORE R6, X       ;X ← R6
```

- The number of Read After Write (RAW) dependencies, Write After Read (WAR) dependencies and Write After Write (WAW) dependencies in the sequence of instruction are, respectively.

A. 2, 2, 4
B. 3, 2, 3
C. ✓ 4, 2, 2
D. 3, 3, 2







- **GATE 2003 CS – For a pipelined CPU with a single ALU, consider the following situations**
  - **I. The j + 1st instruction uses the result of the jth instruction as an operand**
  - **II. The execution of a conditional jump instruction**
  - **III. The jth and j + 1st instruction require the ALU at the same time**
- **Which of the above can cause a hazard**

A. I and II Only
B. II and III Only
C. III Only
D. All of above ✓

① → Data Hazard
② → Control Hazard.
③ → Structural Hazard.

Non Synchronized Pipelining in COA

GATE CS – A four stage pipeline is given below:

	S1	S2	S3	S4
I1	2	1	1	3
I2	1	3	2	1
I3	1	1	3	1
I4	2	1	1	1

$\rightarrow T_p = 13$

$\rightarrow T_{\omega p} = 25$

$\rightarrow S = \frac{T_{\omega p}}{T_p} = \frac{25}{13} = 1.92$

What is the performance gain achieved by the above pipeline structure over non pipeline execution

A. 1.84 B. 1.72 C. 1.92 D. 2.08

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I1	S1	S1	S2	S3	S4	S4	S4							
I2			S1	S2	S2	S2	S3	S3	S4					
I3				S1	–	–	S2	–	S3	S3	S3	S4		
I4					S1	S1	–	S2	–	–	–	S3	S4	

	S1	S2	S3	S4
I1	2	3	4	7
I2	3	6	8	9
I3	4	7	11	12
I4	6	8	12	13

Non Synchronized Pipelining in COA

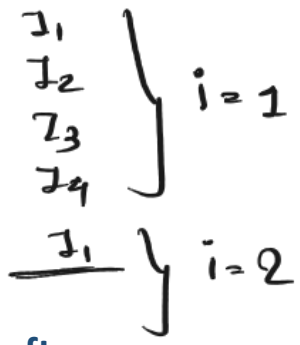
GATE 2004 CS – Consider a pipeline processor with 4 stages S1 to S4. We want to execute the following loop:

For (i=1, i ≤ 1000, i++)

(I1, I2, I3, I4)

Where the time taken (in ns) by instruction I1 to I4 for stages S1 to S4 are given below:

	S1	S2	S3	S4
I1	1	2	1	2
I2	2	1	2	1
I3	1	1	2	1
I4	2	1	2	1



The Output of I1 for i = 2 will be available after

A. 11ns B.12ns C.13ns D. 28ns

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I1	S1	S2	S2	S3	S4	S4								
I2		S1	S1	S2	S3	S3	S4							
I3				S1	S2	–	S3	S3	S4					
I4					S1	S1	S2	–	S3	S3	S4			
I1							S1	S2	S2	–	S3	S4	S4	

	S1	S2	S3	S4
I1	1	3	4	6
I2	3	4	6	7
I3	4	5	8	9
I4	6	7	10	11
I1	7	9	11	13

CPI & Speed Up in Pipelining with Hazard

→ Without Hazard.

$$CPI = \frac{\text{Total Cycles}}{\text{Total Inst.}^n} = \frac{(n+k-1)}{n} = 1 \leftarrow \text{ideal.}$$

→ With Hazard { x cycles due to hazard }.

$$CPI = \frac{(n+k-1)+x}{n} = \boxed{1+\frac{x}{n}} \leftarrow \text{ideal.}$$

→ Without Hazard.

$$S = \frac{T_{wp}}{T_p} = \frac{T_{wp}}{CPI\, T_p}$$

❑ **GATE CS – Consider a 5 stage pipeline which is executing a program of 100 instructions. Among all instructions 10 instructions cause 3 stall cycles each.**

A. **Calculate CPI of Pipeline?**
B. **If Pipeline cycle time is 3ns then what is average instruction execution time?**
C. **Calculate CPI of pipeline in ideal conditions with hazards?**
D. **If pipeline cycle time is 3ns then what is average instruction execution time in ideal conditions?**

→ $k = 5$

$n = 100$

$x = 10 \times 3 = 30$

1] $CPI = \frac{(n+k-1)+x}{n} = \frac{100+5-1+30}{100} = 1.34$

2] $t_{inst^n} = CPI \times t_c = 1.34 \times 3 = 4.02$ nsec

3] $CPI_{ideal} = 1 + \frac{x}{n} = 1 + \frac{30}{100} = 1.3$

4] $t_{inst^n(ideal)} = CPI_{ideal}\, t_c = 1.3 \times 3 = 3.9$ nsec.

Examples on Pipelining Hazards in COA

GATE 2014 CS – Consider a 6 stage instruction pipeline, where all stages are perfectly balanced. Assume that there is no cycle time overhead of pipelining. When an application is executing on this 6 stage pipeline, the speedup achieved with respect to non pipelined execution if 25% of the instructions incur 2 pipeline stall cycles is?

$\rightarrow$ $k = 6$

$\rightarrow$ $CPI = 1 + \frac{x}{n}$

$= 1 + 0.25 \times 2$

$= 1.5$

$\rightarrow$ $S = \frac{T_{wp}}{CPI\ T_p} = \frac{6\ t_c}{1.5\ t_c} = 4$

❑ **GATE CS** – Consider a 3 stage pipeline having delays of 100ns, 200ns and 500ns, respectively. The third stage is spliced into two stages of 250ns and 250ns respectively. Calculate the throughput increase or decrease in percentage.

A. 100% increase
B. 60% increase
C. 60% decrease
D. 50% decrease

P-1 {100ns, 200ns, 500ns}

$k_1 = 3$

$t_{c_1} = 500\,ns$

P-2 {100ns, 200ns, 250ns, 250ns}

$k_2 = 4$

$t_{c2} = 250\,ns$

$\rightarrow TP = \frac{n}{(n+k-1)t_c} = \frac{1}{t_c}$

$\rightarrow TP1 = \frac{1}{t_{c1}} = \frac{1}{500\,ns}$ $\rightarrow TP2 = \frac{1}{t_{c2}} = \frac{1}{250ns}$

$\rightarrow \% = \frac{\frac{1}{250} - \frac{1}{500}}{\frac{1}{500}} = \frac{2-1}{1} \times 100 = 100\%$